

**INSTITUTO TECNOLÓGICO DE BUENOS AIRES
ESCUELA DE GESTIÓN Y TECNOLOGÍA**

Pokemonad

Motor de Juego de Estrategia Multijugador

AUTORES: Francisco Quian Blanco
Theo Stanfield

PROFESORES: Pablo Ernesto Martinez Lopez

TRABAJO FINAL:
72.60 - Programación Funcional - 20252Q

CIUDAD AUTÓNOMA DE BUENOS AIRES
Mayo 2026

Índice

1. Introducción	2
1.1. Contexto y Motivación	2
1.2. Objetivos del Proyecto	2
1.3. Integrantes y Metodología de Trabajo	3
2. Marco Teórico y Conceptos de Programación Funcional	4
2.1. Características del Paradigma a poner en juego	4
2.2. Desafíos en Programación Funcional y Uso de Mónadas	5
2.3. Herramientas y Bibliotecas Utilizadas	6
3. Arquitectura del Motor (Solución Propuesta)	8
3.1. Visión General del Sistema	8
3.2. Gestión del Estado Inmutable	9
3.3. Ciclo de Vida del Juego (Game Loop)	10
4. Detalles de Diseño e Interacción con el Medio	12
4.1. Modelado de Datos	12
4.2. El Rol de las Mónadas en la Interacción	13
4.3. Lógica Multijugador y Concurrencia	15
5. Conclusiones	19
5.1. Evaluación del Alcance y Resultados	19
5.2. Dificultades Encontradas y Resoluciones	19
5.3. Lecciones Aprendidas	20
5.4. Trabajo Futuro	21
5.5. Uso de Herramientas de Inteligencia Artificial	22

1. Introducción

1.1. Contexto y Motivación

El presente trabajo describe el desarrollo de **Pokemonad**, un motor de juego de combate por turnos inspirado en la mecánica clásica de Pokémon, construido íntegramente utilizando el lenguaje de programación funcional Haskell.

La motivación principal del proyecto no radica en la réplica exhaustiva de las complejas reglas del juego original, sino en utilizar este dominio interactivo como vehículo para explorar y resolver problemas de ingeniería de software dentro del paradigma funcional. Tras la etapa de ideación y evaluación de alcance, el enfoque del proyecto se centró en dos grandes desafíos técnicos: la implementación de una arquitectura de red descentralizada Peer-to-Peer (P2P) concurrente y segura, y el desarrollo de una Inteligencia Artificial no trivial para el modo de un solo jugador.

Para lograr la profundidad requerida en estos subsistemas, se tomó la decisión arquitectónica de mantener la lógica de dominio del combate en su forma más pura y básica. Las interacciones se restringieron exclusivamente a acciones de ataque directo (*FIGHT*) y cambio de criaturas (*SWITCH*), omitiendo alteraciones de estado, modificadores de estadísticas y habilidades pasivas. Esta reducción deliberada del alcance permite concentrar el esfuerzo de desarrollo en la gestión del estado concurrente y en los árboles de decisión del bot oponente.

1.2. Objetivos del Proyecto

El objetivo general del trabajo es demostrar la aplicación práctica de conceptos avanzados de programación funcional, tales como la inmutabilidad, la evaluación de funciones puras, y el manejo de efectos a través de mónadas, en la construcción de un sistema de software interactivo y distribuido.

Los objetivos específicos, acordados y delimitados para el alcance de este trabajo, incluyen:

- **Desarrollo de una biblioteca P2P independiente:** Diseñar un módulo de comunicación de bajo nivel mediante sockets TCP puro. Este subsistema gestiona la recepción asincrónica de flujos de bytes (con serialización binaria en tramas) y delega la sincronización de estados concurrentes mediante Memoria Transaccional en Software (STM, usando `TQueue`). Este módulo opera de forma completamente desacoplada de las reglas de *Pokemonad*, constituyendo una biblioteca genérica.

- **Implementación de un oponente controlado por IA:** Construir un agente inteligente para el modo de un jugador que supere la toma de decisiones heurística trivial. Se implementó un modelo basado en **Aprendizaje por Refuerzo (Reinforcement Learning)**, utilizando una aproximación lineal para estimar los valores Q (Q-Values) a partir de la extracción de características del entorno (ventaja de tipos, proyecciones de daño y viabilidad de cambio de criaturas). La selección de acciones se rige por una política ϵ -greedy parametrizada según la dificultad del oponente.
- **Integración de una Interfaz Gráfica (GUI):** Proveer una capa visual interactiva utilizando la biblioteca `gloss`. Esta herramienta permite renderizar gráficos 2D de forma declarativa, exponiendo el estado inmutable del motor en tiempo real e integrándose fluidamente con el bucle principal de la aplicación.

1.3. Integrantes y Metodología de Trabajo

El proyecto fue diseñado y desarrollado íntegramente por Francisco Quian Blanco y Theo Stanfield.

Dada la complejidad inherente de orquestar un hilo de red asincrónico junto con un hilo de interfaz gráfica (UI) evitando condiciones de carrera, se adoptó una metodología de desarrollo colaborativa y altamente comunicativa. La división de tareas se gestionó de manera dinámica apoyándose en un sistema de control de versiones.

Para los componentes críticos de la arquitectura —particularmente el aislamiento de la biblioteca P2P y la implementación de la IA— se priorizó el trabajo conjunto mediante sesiones de *pair programming*. Para el desarrollo de módulos individuales, se establecieron revisiones de código cruzadas (*code reviews*). Esta metodología garantiza no solo el avance equilibrado del proyecto, sino que asegura que ambos integrantes mantengan un conocimiento total y un manejo profundo de cada aspecto del código fuente producido, cumpliendo rigurosamente con los requisitos de evaluación.

2. Marco Teórico y Conceptos de Programación Funcional

El desarrollo de un motor interactivo como **Pokemonad** en Haskell exige aplicar los pilares del paradigma funcional puro para garantizar un código predecible, testeable y seguro [3].

2.1. Características del Paradigma a poner en juego

Las características centrales puestas en juego son:

- **Inmutabilidad:** A diferencia de los lenguajes imperativos donde el estado de un objeto se sobrescribe en memoria, en Haskell las estructuras de datos son inmutables. En el proyecto, cuando un Pokémon recibe daño, no se modifica su variable interna de puntos de vida. En su lugar, el módulo `Pokemonad.Battle.State` genera una copia completamente nueva del estado de la batalla que refleja la reducción de salud, preservando el estado anterior de forma intacta.
- **Funciones Puras:** Una función pura garantiza que, para los mismos argumentos de entrada, siempre retornará el mismo resultado sin producir efectos secundarios. Los cálculos delegados a `Pokemonad.Battle.Logic` y `Pokemonad.Battle.Damage` son puros; el daño resultante de un ataque depende estrictamente de las estadísticas del atacante, del defensor y de las propiedades del movimiento, lo que permite razonar matemáticamente sobre el combate y facilita enormemente las pruebas unitarias.
- **Tipos de Datos Algebraicos (ADTs):** Los ADTs permiten modelar el dominio del problema de forma expresiva y segura en tiempo de compilación. A través de módulos como `Pokemonad.Core.Types`, `Pokemonad.Core.Move` y `Pokemonad.Core.Pokemon`, el sistema representa exhaustivamente los tipos elementales, los movimientos y los estados de los entrenadores (`Pokemonad.Core.Trainer`), evitando estados inválidos o transiciones ilógicas.
- **Funciones de Orden Superior y Composición:** La capacidad de pasar funciones como parámetros o retornarlas permite construir lógicas complejas a partir de piezas simples. En la evaluación de la Inteligencia Artificial (`Pokemonad.AI.Model` y `Pokemonad.AI.Decision`), las funciones de orden superior extraen y ponderan características del entorno de forma declarativa para construir árboles de decisión limpios [2].

- **Evaluación Perezosa (Lazy Evaluation):** Haskell no evalúa una expresión hasta que su resultado es estrictamente necesario. En el contexto de la simulación de turnos o en la generación de ramificaciones de decisiones para la IA, la evaluación perezosa permite definir espacios de estados potencialmente gigantescos sin incurrir en costos de memoria o procesamiento inmediatos, computando únicamente el camino que el motor decide explorar.

2.2. Desafíos en Programación Funcional y Uso de Mónadas

Construir un videojuego interactivo y multijugador bajo el rigor funcional impone desafíos de diseño, fundamentalmente al lidiar con mutabilidad, interacción externa y concurrencia. Para abordar esto, `Pokemonad` implementa un patrón de diseño conocido como “Pure Core + Thin IO Layer”, separando estrictamente el `game-engine` (núcleo puro) del `game-client` y `p2p-net` (capas de interacción).

Las herramientas fundamentales para resolver estos desafíos son las **mónadas**, las cuales encapsulan el contexto computacional de las operaciones:

- **Modelado del Estado Dinámico (Mónadas State y Transformadores StateT):** Simular el cambio de estado turno a turno requiere propagar estructuras complejas. La biblioteca `mtl` [1] permite utilizar `State` o `StateT` para enhebrar implícitamente el contexto del juego (`Pokemonad.Battle.State` y `Client.State`) a través de las operaciones. Esto otorga la conveniencia sintáctica de la mutabilidad secuencial sin sacrificar la pureza matemática subyacente.
- **Interacción con el Medio (Mónada IO):** Todo efecto secundario (dibujar en pantalla, leer el teclado o acceder al disco) debe confinarse en `IO`. La capa visual manejada en `Client.Render` y `Client.Drawing`, así como la lectura de archivos de entrenamiento como `data/ai_checkpoint.txt`, ocurren dentro de este contexto asilado, protegiendo al motor puro de inconsistencias.
- **Concurrencia y Sincronización Multijugador (Mónada STM):** El mayor desafío del multijugador P2P es gestionar la memoria compartida entre el hilo de red (escuchando sockets) y el hilo gráfico (renderizando el juego) sin incurrir en condiciones de carrera. El proyecto utiliza `STM` (Software Transactional Memory) [4] para garantizar que los intercambios de estado y eventos en la red ocurran de manera atómica, coordinada y sin bloqueos explícitos (deadlocks).

- **Aleatoriedad en un Entorno Puro:** La generación de eventos probabilísticos requiere romper el determinismo trivial. Esto se resuelve pasando explícitamente generadores pseudoaleatorios (`random`) a través de las funciones puras en `Pokemonad.Battle.Turn` y los módulos de decisión de la IA, garantizando que la simulación completa sea reproducible.
- **Manejo de Errores y Fallos (Mónadas `Either` y `Maybe`):** Para la comunicación en red a través de `P2P.Serialization` y `Client.NetSerializers`, el parseo de bytes entrantes puede fallar. En lugar de lanzar excepciones que rompan el flujo de ejecución, se utilizan contextos como `Either` o `Maybe` para representar de forma explícita y segura el éxito o fracaso de la decodificación.

2.3. Herramientas y Bibliotecas Utilizadas

Para materializar esta arquitectura, el proyecto se apoya en un ecosistema robusto gestionado mediante **Cabal** [5], la herramienta estándar de Haskell para definir dependencias, compilar y organizar la solución en tres paquetes interconectados (`game-engine`, `p2p-net` y `game-client`).

- **network:** Provee el acceso de bajo nivel a los sockets TCP en `P2P.Communication`, estableciendo los canales directos requeridos para la arquitectura Peer-to-Peer.
- **stm:** Fundamental para la arquitectura asincrónica. Proporciona las primitivas de memoria transaccional para coordinar de forma segura el estado del cliente y los mensajes recibidos a través de la red.
- **binary y bytestring:** Utilizados en `P2P.Serialization` y `Client.NetSerializers`. Permiten manipular secuencias de bytes crudas y proveen las clases de tipos necesarias para empaquetar y desempaquetar eficientemente los mensajes del protocolo para su envío por la red.
- **mtl (Monad Transformer Library):** Extensión esencial empleada exhaustivamente en el `game-engine` para combinar mónadas sin acoplar lógicas de infraestructura a la lógica de negocio.
- **random:** Implementa la lógica de generación de números pseudoaleatorios, indispensable tanto para la incertidumbre del combate como para las dinámicas de exploración en el entrenamiento de la Inteligencia Artificial (`Pokemonad.AI.Training`).

- **gloss y gloss-juicy**: Ecosistema gráfico funcional elegido para **game-client**. **gloss** permite renderizar escenas 2D de forma puramente declarativa conectándolas a modelos de estado, mientras que **gloss-juicy** extiende estas capacidades permitiendo cargar y manipular imágenes rasterizadas complejas alojadas en la carpeta **assets**.
- **containers y text**: Proporcionan estructuras de datos eficientes (como mapas) utilizadas para indexar elementos, y permiten manejar cadenas de caracteres Unicode con alta eficiencia de memoria en la interfaz, superando a las listas de caracteres nativas.

La elección de programación funcional pura, soportada por mónadas transaccionales y de estado, garantiza que la complejidad inherente a un sistema multijugador concurrente y de comportamiento no determinista permanezca manejable. Estas bases teóricas sientan el terreno propicio para comprender la división arquitectónica y las decisiones de diseño estructural de las siguientes secciones.

3. Arquitectura del Motor (Solución Propuesta)

3.1. Visión General del Sistema

La arquitectura de Pokemonad se organiza en tres paquetes de Cabal [5] con responsabilidades estrictamente delimitadas, reflejando una decisión de diseño tomada desde el inicio del proyecto: hacer explícita y verificable la separación entre la lógica pura del dominio, la comunicación en red y la presentación visual.

- **game-engine:** Actúa como la “calculadora” del sistema. Contiene la totalidad del conocimiento de dominio del combate: la base de datos de los 151 Pokémon originales, sus 197 movimientos, las estadísticas por especie, los entrenadores predefinidos, el cálculo de daño, la resolución de turnos y el motor de Inteligencia Artificial. Sus módulos son en su gran mayoría funciones puras sin efectos secundarios, lo que los hace completamente predecibles, reproducibles y verificables de forma aislada.
- **p2p-net:** Implementa una biblioteca de comunicación Peer-to-Peer genérica y agnóstica al juego. Gestiona los canales TCP de bajo nivel, la serialización binaria de mensajes mediante tramas enmarcadas, y expone las estructuras de memoria transaccional para la sincronización entre hilos. Este paquete desconoce completamente las reglas de Pokémon; podría reutilizarse para cualquier aplicación que requiera comunicación P2P concurrente.
- **game-client:** Es el puente que orquesta los dos paquetes anteriores. Integra la interfaz gráfica construida con **gloss**, consume la API pura del **game-engine** para aplicar acciones y obtener nuevos estados de combate, y utiliza **p2p-net** para sincronizar el estado de la partida con el jugador remoto. Es la única capa que tiene conocimiento tanto de las mecánicas del juego como del protocolo de red.

Esta separación en tres paquetes fue una decisión arquitectónica deliberada, no el resultado orgánico del crecimiento del proyecto. El objetivo fue hacer explícitos y no triviales los dos subsistemas de mayor complejidad técnica —la red P2P y la IA— garantizando que cada paquete tuviera una única razón de cambio. Esta estructura se corresponde con el patrón *Pure Core + Thin IO Layer* mencionado en la sección anterior: **game-engine** es el núcleo puro, mientras que **game-client** y **p2p-net** conforman la capa de interacción con el entorno.

3.2. Gestión del Estado Inmutable

El estado global del sistema reside en el registro `World`, definido en `Client.State` dentro del paquete `game-client`:

```
data World = World
  { worldGame      :: AppState
  , netInQueue     :: TQueue AppMsg
  , netSubState    :: NetSubState
  , netSocket      :: Maybe Socket
  , netConnAsync   :: TVar (Maybe NetConnAsync)
  , aiTrainingAsync :: TVar (Maybe AITrainingResult)
  , netIsHost      :: Bool
  }
```

El campo central `worldGame :: AppState` concentra el estado lógico de la pantalla actual, el progreso de la partida y el `BattleState` activo. Su gestión sigue estrictamente el principio de inmutabilidad: ninguna función modifica el registro `World` en su lugar. En cambio, cada manejador de evento o de tick recibe el `World` actual y devuelve una copia nueva con los campos actualizados. El estado anterior queda intacto y el flujo de información es unidireccional y predecible.

La gestión de la aleatoriedad dentro del núcleo puro merece mención particular. En lugar de recurrir a la mónada `State StdGen` o a una interfaz de aleatoriedad implícita como `MonadRandom`, el motor adopta la técnica de *threading explícito del generador*: cada función que requiere aleatoriedad recibe un `StdGen` como parámetro adicional y devuelve el generador consumido junto con su resultado. Un ejemplo representativo es la función de resolución de turno:

```
executeTurn :: StdGen -> BattleState -> BattleAction
            -> BattleAction -> ([BattleStep], StdGen)
```

Esta decisión, motivada por el deseo de no oscurecer el flujo de efectos, tiene una consecuencia directa sobre la legibilidad: la firma de la función hace explícito que consume aleatoriedad y produce un nuevo generador, sin ningún efecto oculto. Refleja fielmente el principio de inmutabilidad: el `StdGen` original no se modifica; a partir de él se genera y retorna uno nuevo.

Los campos `TVar` y `TQueue` del registro `World` son los únicos puntos de estado verdaderamente mutable del sistema. Sin embargo, esta mutabilidad está estrictamente controlada: solo puede modificarse dentro de una transacción `atomically`, garantizando consistencia bajo concurrencia. El resto del

`World` es un valor Haskell ordinario, inmutable y copiado funcionalmente en cada transformación.

3.3. Ciclo de Vida del Juego (Game Loop)

El bucle principal del motor está gestionado por la función `playIO` de la biblioteca `gloss`, que actúa como el *runtime* de la aplicación:

```
playIO window black 30 world0 drawWorld handleWorldInput handleWorldTick
```

Esta función recibe el estado inicial (`world0`), la función de renderizado (`drawWorld :: World -> IO Picture`), el manejador de eventos de entrada (`handleWorldInput :: Event -> World -> IO World`) y el manejador de avance de tiempo (`handleWorldTick :: Float -> World -> IO World`). La elección de `playIO` sobre la variante pura `play` fue necesaria para poder ejecutar operaciones en las mónadas `IO` y `STM` dentro de los manejadores de tick, en particular para consultar las `TVar` y drenar la `TQueue` de mensajes de red de forma atómica.

Pokemonad opera con hasta tres hilos de ejecución concurrentes:

1. **Hilo principal (Gloss, 30 Hz):** Ejecuta el renderizado y los manejadores de eventos y tick. Es el único productor y consumidor del registro `World`.
2. **Hilo de red:** Lanzado mediante `forkIO` en `P2P.Communication.forkRecvLoop` cuando se establece una conexión. Bloquea continuamente sobre el socket TCP esperando datos, decodifica las tramas recibidas y escribe cada `AppMsg` decodificado en la `TQueue` de forma atómica.
3. **Hilo de entrenamiento de IA (opcional):** Lanzado mediante `forkIO` en `Client.Handlers.AISimulatorHandler` cuando el usuario inicia un ciclo de entrenamiento. Ejecuta el proceso de auto-juego (*self-play*) de Q-Learning de forma completamente asincrónica y escribe el resultado en la `TVar` correspondiente al finalizar.

En cada tick de 30 Hz, `handleWorldTick` ejecuta una secuencia determinista de pasos para integrar toda la información disponible en el nuevo estado del mundo:

1. `mergeNetAsync`: consulta la `TVar` de resultado de conexión y, si está disponible, inicializa el subsistema de red y lanza el hilo de recepción.

2. `drainNetInbox`: consume de forma atómica *todos* los mensajes pendientes en la `TQueue` y los aplica secuencialmente al estado del mundo mediante `foldl'`.
3. `mergeMultiplayerLobby` / `mergeMultiplayerBattle`: procesa las transiciones de estado del protocolo multijugador y, si corresponde, ejecuta el siguiente turno de combate.
4. `mergeAITraining`: consulta la `TVar` de resultado de entrenamiento y, si está disponible, integra los nuevos pesos del modelo al estado de la aplicación.
5. `handleTick`: avanza las animaciones de batalla (sacudidas de sprites, desvanecimientos, desplazamiento del log de mensajes).

El uso de `foldl'` (la variante estricta de `foldl`) en el paso de drenado de mensajes es un ejemplo concreto del control manual de la evaluación en Haskell: dado que Haskell es perezoso por defecto, acumular un *fold* sobre una lista larga sin forzar la evaluación puede generar fugas de espacio (*space leaks*) por la acumulación de *thunks* no evaluados. La forma estricta fuerza la evaluación del acumulador en cada paso, garantizando comportamiento predecible en memoria.

4. Detalles de Diseño e Interacción con el Medio

4.1. Modelado de Datos

Los Tipos de Datos Algebraicos (ADTs) de Haskell son la herramienta central para representar el dominio del juego de forma precisa y segura en tiempo de compilación. A diferencia de las enumeraciones simples o las jerarquías de clases de los lenguajes orientados a objetos, los ADTs permiten expresar con exactitud qué valores son posibles en cada punto del sistema, y el compilador verifica exhaustivamente que todos esos casos sean tratados.

El ADT más representativo del diseño es `BattlePhase`, que codifica la máquina de estados finita del combate:

```
data BattlePhase
  = WaitingForCommand
  | TurnExecution
  | WaitingForForcedPlayerSwitch
  | WaitingForForcedEnemySwitch
  | BattleEnded Winner
```

Cada variante representa un estado legítimo y distinto del combate. El compilador garantiza que toda función que opere sobre `BattlePhase` deba manejar explícitamente cada constructor mediante *pattern matching* exhaustivo. Si en el futuro se añade un nuevo estado al sistema, el compilador señalará todas las funciones que requieren actualización, convirtiendo un potencial error en tiempo de ejecución en un error en tiempo de compilación. Esta característica fue una de las más valoradas por los autores durante el desarrollo: proporciona garantías que en otros lenguajes solo se obtienen mediante pruebas extensas o convenciones manuales.

El patrón de *guards* (1) complementa al *pattern matching* para estructurar funciones con múltiples condiciones, evitando el anidamiento de `if-else` y acercando la lectura del código a la notación matemática por casos:

```
continuePlayerCheck bState logs
  | unHP (battlePokemonHp (playerActive bState)) <= 0 =
    case firstAliveIndex (playerBench bState) of
      Just _ -> (bState { phase = WaitingForForcedPlayerSwitch }, ...)
      Nothing -> (bState { phase = BattleEnded EnemyWon }, ...)
  | otherwise = (bState { phase = WaitingForCommand }, logs)
```

El patrón de *newtype wrappers* fue adoptado extensivamente en `Pokemonad.Core.Types` para resolver una limitación concreta de Haskell: la ausencia de parámetros nombrados en las funciones. Sin este recurso, una función como `calculateDamage` con múltiples parámetros enteros resulta ambigua e ilegible en el sitio de llamada:

```
-- Sin newtype: ¿qué es cada número?
applyAction PlayerSide rng bState (ActionMove 2)

-- Con newtype: la semántica de cada argumento es verificada por el compilador
newtype HP          = HP          { unHP          :: Int }
newtype PokemonId   = PokemonId   { unPokemonId   :: Int }
newtype Level        = Level        { unLevel        :: Int }
```

Estas definiciones no generan overhead en runtime (Haskell las elimina en compilación mediante *newtype deriving*), pero previenen errores de transposición de argumentos que de otro modo serían silenciosos.

Durante el diseño también se evaluó la posibilidad de modelar la lógica de combate como una *Free Monad*, separando la sintaxis del árbol de decisiones de su interpretación. Esta alternativa fue descartada porque el **game-engine**, concebido como una calculadora de estado pura, ya cumplía el objetivo de separación de responsabilidades sin la complejidad adicional. La simplicidad de funciones puras con paso explícito de estado resultó suficiente y más legible para el alcance del proyecto.

4.2. El Rol de las Mónadas en la Interacción

Toda interacción de `Pokemonad` con el entorno —pantalla, red, disco, concurrencia— está mediada por mónadas. El diseño concentra todos los efectos secundarios en la capa **game-client** y en el módulo `Pokemonad.AI.Persistence`, manteniendo el núcleo del **game-engine** completamente puro: 13 de sus 14 módulos no contienen ninguna operación IO.

La *do*-notation de Haskell es el mecanismo sintáctico que permite expresar secuencias de operaciones monádicas de forma legible. Una expresión como:

```
do
  x <- operacionA
  y <- operacionB x
  pure (combinar x y)
```

es azúcar sintáctica equivalente a:

```
operacionA >>= \x -> operacionB x >>= \y -> pure (combinar x y)
```

El operador `>>=` (*bind*) secuencia operaciones pasando el resultado de cada una a la siguiente, delegando en cada mónada concreta la semántica de esa composición. La potencia de este mecanismo es que el mismo patrón sintáctico funciona de forma uniforme en contextos radicalmente distintos:

- **Mónada IO:** Cada `x <- accion` ejecuta un efecto en el mundo real (leer un sprite del disco, escribir en un socket, inicializar un generador pseudoaleatorio) y vincula el resultado al nombre `x`. El sistema de tipos garantiza que el código IO no pueda “escapar” de su contexto: una función con tipo de retorno IO a solo puede ser invocada desde otra función IO. Esta propiedad es la que permite al compilador garantizar que `game-engine` está libre de efectos secundarios. Los usos concretos incluyen el bucle principal (`Main.hs`), los manejadores de `gloss` (`Client.Events`), la comunicación de red (`P2P.Communication`) y la persistencia de pesos de la IA (`Pokemonad.AI.Persistence`).
- **Mónada STM:** Dentro de un bloque `atomically`, la `do`-notation opera sobre STM, cuya semántica de `>>=` es transaccional: si alguna operación dentro del bloque detecta un conflicto con otro hilo, la transacción entera se descarta y se reintenta automáticamente. Esto elimina la posibilidad de *deadlocks* por inversión de orden de adquisición de locks. El uso concreto en el proyecto es el drenado atómico de la cola de mensajes de red:

```
drainAll :: TQueue a -> STM [a]
drainAll q = do
  mx <- tryReadTQueue q
  case mx of
    Nothing -> pure []
    Just x   -> fmap (x:) (drainAll q)
```

Esta función recorre la `TQueue` hasta vaciarla, y toda la secuencia de lecturas es atómica desde la perspectiva del hilo de red: o se leen todos los mensajes disponibles o ninguno.

- **Mónadas Maybe y Either:** Utilizadas en el núcleo puro para representar operaciones que pueden carecer de resultado o que pueden fallar con un mensaje de error. Por ejemplo, `switchActive` en `Pokemonad.Battle.Logic` retorna `Either String BattleState`: si el Pokémon seleccionado ya

está activo o no existe, el resultado es `Left "mensaje de error"` sin lanzar ninguna excepción. Este estilo convierte los casos de error en valores de primer orden que el compilador obliga a manejar explícitamente.

La distinción fundamental entre `IO` y `STM` ilustra el poder del sistema de tipos de Haskell: no es posible ejecutar una operación `STM` fuera de un `atomically`, y no es posible llamar a una función `IO` dentro de un bloque `STM`. El compilador verifica estas invariantes estáticamente, previniendo una clase entera de errores de concurrencia antes de que el programa sea ejecutado.

4.3. Lógica Multijugador y Concurrencia

Modelo Host-Árbitro / Cliente-Receptor

La arquitectura multijugador adopta un modelo **host-árbitro / cliente-receptor**: el jugador que inicia la sesión (*host*) es la única fuente de verdad del estado del combate. Ambas instancias conocen las mecánicas del juego, pero solo el host ejecuta los turnos. Esta decisión simplifica radicalmente el protocolo al eliminar la necesidad de reconciliar dos estados divergentes.

Serialización del Protocolo

El protocolo de comunicación se define mediante el ADT `AppMsg` en `P2P.Types`:

```
data AppMsg
  = AppMsgHandshake Handshake
  | AppMsgTeam [Word32]
  | AppMsgBattleReady
  | AppMsgAction PlayerAction
  | AppMsgBattleState ByteString
  | AppMsgBattleFrames ByteString
  | AppMsgDisconnect
```

La serialización utiliza la biblioteca `binary`, que expone la clase de tipos `Binary` para empaquetar y desempaquetar valores de Haskell a secuencias de bytes. El módulo `P2P.Serialization` implementa el encuadre de mensajes (*framing*): cada mensaje se precede de su longitud en bytes, permitiendo reconstruir tramas completas desde el flujo continuo de TCP.

El Patrón de Perspectiva Invertida

Un desafío sutil del protocolo surge del hecho de que el `BattleState` siempre está definido desde la perspectiva del jugador local: `playerActive` es invariante quien usa la pantalla actual. Cuando el host ejecuta un turno, los frames resultantes están orientados desde *su* perspectiva; enviarlos directamente al cliente produciría una representación invertida donde el cliente vería a su rival como “jugador”.

Para resolver esto, el motor define las funciones `flipBattleState` y `flipBattleStep`, que intercambian los campos de jugador y enemigo en el estado y en cada frame de animación. El host aplica este flip antes de serializar y enviar los frames:

```
let framesForPeer = map flipBattleStep frames
    encoded = BL.toStrict (encode framesForPeer)
sendMsg sock (AppMsgBattleFrames encoded)
```

El cliente permanece completamente ignorante de que existe una perspectiva opuesta: siempre recibe los frames ya orientados correctamente y simplemente los renderiza. Este diseño mantiene al cliente en un rol pasivo y concentra toda la complejidad de coordinación en el host.

Manejo del Cambio Forzado

El flujo estándar del combate es simétrico: ambos jugadores envían una acción, el host ejecuta el turno y distribuye los frames resultantes. Sin embargo, cuando el Pokémon activo de un jugador se desmaya, ese jugador debe elegir un reemplazo *antes* de que el siguiente turno pueda resolverse. Esta asimetría introduce la mayor complejidad del protocolo multijugador.

La solución se articula a través de `BattlePhase` como máquina de estados: cuando `resolveTurnAfterDamageMulti` detecta un Pokémon con HP en cero y al menos un sustituto disponible en el banco, transiciona el estado a `WaitingForForcedPlayerSwitch` o `WaitingForForcedEnemySwitch` según corresponda. Este estado es serializado como parte del `BattleState` y transmitido al cliente mediante `AppMsgBattleState`.

Al recibir el estado, el cliente decodifica la fase y el sistema de UI automáticamente impone el menú de selección de Pokémon, impidiendo cualquier otra acción. En el host, la función `mergeMultiplayerBattle` solo procede a ejecutar el turno cuando la acción correspondiente a la fase está disponible, retornando el mundo sin cambios en caso contrario:

```
WaitingForForcedEnemySwitch ->
```

```

case battlePendingRemoteAction bss of
  Just remoteAction -> executeMPTurn ...
  Nothing           -> pure w    -- aguarda sin bloquear el hilo de UI

```

Este diseño convierte el tick de `gloss` en un bucle de encuesta (*polling*) eficiente: el host revisa en cada frame si las condiciones para ejecutar el turno están satisfechas, sin bloquear el hilo principal.

Inteligencia Artificial: Q-Learning con Auto-Juego

Para el modo de un jugador se evaluaron tres enfoques para el agente oponente:

1. **Heurística simple:** Ordenar los ataques disponibles por efectividad y seleccionar con aleatoriedad inversamente proporcional a la dificultad. Simple de implementar, pero incapaz de generalizar estrategias no evidentes como la gestión del banco de Pokémon.
2. **Minimax:** Exploración del árbol de estados del combate con función de evaluación en los nodos hoja. Descartado por la explosión combinatoria: el espacio de estados de un combate Pokémon es demasiado amplio para hacer practicable la exploración exhaustiva por árbol.
3. **Q-Learning con aproximación lineal:** Aprendizaje por refuerzo que estima el valor de las acciones a partir de características del estado. Seleccionado por su capacidad de generalizar estrategias complejas, sus garantías de convergencia conocidas y su complejidad de implementación adecuada al alcance del proyecto. La familiaridad de los autores con técnicas de aprendizaje por refuerzo, adquirida en la materia Sistemas de Inteligencia Artificial, influyó también en esta decisión.

El modelo representa el valor esperado de tomar una acción a en un estado s como una combinación lineal de características:

$$Q(s, a) = b + \mathbf{w} \cdot \phi(s, a)$$

donde b es un sesgo escalar, $\mathbf{w}$ es el vector de pesos aprendibles y $\phi(s, a)$ es el vector de 10 características extraídas del estado. La elección de aproximación lineal sobre una red neuronal se justifica por la complejidad de entrenamiento: para el espacio de acción reducido del proyecto (solo *FIGHT* o *SWITCH*) y el conjunto de características ya significativas, la linealidad es suficiente, más interpretable y más estable de entrenar con la regla de diferencia temporal.

Las 10 características fueron diseñadas por intuición del dominio del juego (*feature engineering*):

- Razón de HP propio y del enemigo (posición de ventaja global).
- Daño esperado normalizado y efectividad de tipo del movimiento disponible.
- Probabilidad estimada de noquear al rival en este turno.
- Ventaja de velocidad (determina quién ataca primero en el turno).
- Ganancia ofensiva y defensiva de realizar un cambio de Pokémon.
- Priors de acción (*fight* vs. *switch*) para capturar sesgos base de estrategia.

Un ejemplo del razonamiento de dominio detrás de estas features: cambiar de Pokémon constantemente puede reducir el daño recibido, pero implica desperdiciar un turno de ataque. Las features de ganancia defensiva de cambio y el prior de acción de *switch* capturan este trade-off, permitiendo que el modelo aprenda cuándo vale la pena sacrificar un turno de ataque.

El entrenamiento utiliza *self-play*: dos instancias del modelo con los mismos pesos compiten entre sí. Los pesos se actualizan con la regla de diferencia temporal (TD):

$$\delta = r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot \delta \cdot \phi(s, a)$$

donde r es la recompensa inmediata, γ el factor de descuento y α la tasa de aprendizaje. La selección de acciones sigue una política ϵ -greedy cuyo parámetro varía según la dificultad: desde un 35 % de exploración en dificultad fácil hasta 0 % en el nivel extremo.

Todo el ciclo de entrenamiento (`Pokemonad.AI.Training`) es completamente puro: el `StdGen` se enhebra explícitamente a través de cada episodio de auto-juego, y los pesos se actualizan mediante funciones `tdUpdate :: ... -> QWeights -> QWeights` sin ningún efecto secundario. El único contacto con IO ocurre en `Pokemonad.AI.Persistence`, que serializa los pesos aprendidos al disco al finalizar el entrenamiento.

5. Conclusiones

5.1. Evaluación del Alcance y Resultados

Pokemonad cumple con el alcance comprometido en la propuesta aprobada. Los dos subsistemas que justificaban la complejidad del proyecto fueron implementados de forma no trivial y verificable: la arquitectura P2P concurrente mediante un protocolo de sincronización basado en STM con gestión explícita de fases de combate, y el agente de Inteligencia Artificial mediante un modelo de Q-Learning con aproximación lineal entrenado por auto-juego, superando los enfoques heurístico y Minimax originalmente contemplados en la propuesta.

La simplificación deliberada de las mecánicas de combate —únicamente acciones *FIGHT* y *SWITCH*, sin efectos de estado, modificadores de estadísticas ni habilidades pasivas— no constituye una limitación accidental sino una decisión de alcance consciente. En fases tempranas del desarrollo se contaba con una implementación parcial de estas mecánicas adicionales, pero su integración con la lógica de sincronización multijugador y con el vector de características de la IA introducía una complejidad desproporcionada respecto al valor que aportaban. La premisa central del proyecto es el combate Pokémon como vehículo para un motor funcional complejo, no como una réplica fiel del juego original.

La propuesta inicial también contemplaba el uso de *Free Monads* para modelar la lógica de combate y la biblioteca `lens` para manipulación de estructuras anidadas. Ambas fueron descartadas durante el desarrollo: las *Free Monads* resultaron innecesarias dado que el engine, concebido como calculadora pura, ya ofrecía la separación de responsabilidades buscada; `lens` fue reemplazada por accesorios explícitos que resultaron más legibles para el dominio concreto del proyecto.

5.2. Dificultades Encontradas y Resoluciones

Diseño de la Inteligencia Artificial. El mayor esfuerzo intelectual del proyecto fue anterior al código: la elección y comprensión del algoritmo de aprendizaje, el diseño del vector de características y la construcción del ciclo de auto-juego. Entender por qué la aproximación lineal es preferible a una Q-table completa (espacio de estados intratable) o a una red neuronal (complejidad de entrenamiento desproporcionada para el dominio), y diseñar características que capturen adecuadamente la dinámica del combate, requirió investigación y múltiples iteraciones conceptuales antes de escribir una sola línea de implementación.

Cambio forzado en el protocolo multijugador. El flujo estándar del combate es simétrico: ambos jugadores envían una acción y el host ejecuta el turno. El cambio forzado —cuando un Pokémon se desmaya y debe ser reemplazado antes del siguiente turno— rompe esta simetría e introduce un estado de espera asimétrico. Modelarlo correctamente requirió diseñar `BattlePhase` como una máquina de estados finita explícita con variantes de cambio forzado, adaptar el protocolo de mensajes para transmitir la fase del estado al cliente, y ajustar la lógica del host para esperar únicamente la acción correspondiente a la fase activa.

Curva de aprendizaje de Haskell. La sintaxis de Haskell es densa e inicialmente opaca para desarrolladores con experiencia previa en lenguajes imperativos. Operadores como `$`, `.`, `>>=` o `<$>`, la notación `do`, los *guards* y el sistema de tipos avanzado tienen una curva de ingreso significativa. Esta dificultad disminuye notablemente una vez que se comprenden los principios subyacentes, pero representa una barrera real de entrada al ecosistema que demanda tiempo de adaptación.

Ausencia de parámetros nombrados. Haskell carece de la capacidad de nombrar argumentos en el sitio de llamada (a diferencia de Python o Swift), lo que hace que funciones con múltiples parámetros del mismo tipo sean propensas a errores silenciosos de transposición. La solución adoptada fue el uso sistemático de *newtype wrappers* en `Pokemonad.Core.Types` para dotar de semántica verificable a cada argumento, resolviendo el problema sin costo en runtime.

5.3. Lecciones Aprendidas

El *pattern matching* exhaustivo es la característica más valiosa del paradigma. La garantía del compilador de que todos los casos de un ADT están cubiertos previene en tiempo de compilación una categoría entera de errores que en otros lenguajes solo se detectan en tiempo de ejecución. Cada vez que se incorporó un nuevo `BattlePhase` o un nuevo `AppMsg` al protocolo, el compilador señaló exactamente qué funciones requerían actualización, convirtiendo la extensibilidad del sistema en un proceso guiado y seguro.

La evaluación perezosa habilita estilos de código más claros sin sacrificar performance. Durante los últimos refactors del proyecto se simplificaron bloques con profundidad excesiva de `let` anidados adoptando un patrón de “asignar todo, computar al final”: todos los valores intermedios se definen con nombres descriptivos en el bloque `let`, y la lógica final se expresa en el `in` usando esos nombres. La evaluación perezosa garantiza que los valores no utilizados en una rama particular de la lógica no generen costo

computacional, haciendo este estilo más legible sin penalización alguna de rendimiento.

La separación de núcleo puro y capa de efectos simplifica el razonamiento. El patrón *Pure Core + Thin IO Layer*, materializado en la separación de los tres paquetes de Cabal, resultó en un sistema cuyo motor de combate puede razonarse y verificarse de forma aislada. Las funciones de **game-engine** son completamente deterministas dado un **StdGen** inicial, haciendo que la lógica de combate sea reproducible sin necesidad de levantar ninguna infraestructura de red o interfaz gráfica.

STM es preferible a la sincronización manual con locks. La gestión de la concurrencia mediante transacciones atómicas elimina la posibilidad de *deadlocks* por inversión de orden de adquisición de locks. La garantía de atomicidad compositiva de STM —múltiples operaciones en un bloque **atomically** son atómicas como unidad— simplificó significativamente el diseño del protocolo de sincronización entre el hilo de red y el hilo de UI.

5.4. Trabajo Futuro

De continuar el desarrollo, las extensiones más naturales serían:

- **Mecánicas de combate completas.** La incorporación de efectos de estado (parálisis, quemadura, sueño), modificadores de estadísticas (*buffs* y *debuffs*), habilidades pasivas e ítems de la mochila es la extensión más obvia. Su impacto en el sistema actual sería amplio: requeriría extender el **BattleState**, enriquecer el vector de características de la IA y actualizar el protocolo de sincronización multijugador.
- **Reconexión ante desconexiones de red.** El manejo actual de errores de red es simple: si la conexión cae inesperadamente, el jugador remoto sale con un error. Una desconexión limpia —mediante la acción de salir por el menú— sí envía un mensaje explícito **AppMsgDisconnect** y ambos cierran el socket de forma ordenada. Un sistema de reconexión requeriría persistir el estado del combate en disco y reestablecerlo al reconectarse.
- **Entrenamiento continuo en partida.** La IA actualmente se entrena en batch antes de iniciar una partida. Una extensión interesante sería el aprendizaje en tiempo real durante el combate, actualizando los pesos del modelo tras cada turno observado mediante el hilo de entrenamiento de IA y comunicando el resultado al hilo de UI mediante la **TVar** ya existente. Esta posibilidad fue descartada en el alcance actual por añadir complejidad desproporcionada, y porque la arquitectura P2P

sin servidor central no ofrece un punto centralizado donde entrenar el modelo de forma compartida entre partidas.

- **Soporte para más de dos jugadores.** La arquitectura P2P actual es punto a punto. Un modo torneo requeriría una topología en estrella o en malla, con el host coordinando múltiples conexiones simultáneas mediante un conjunto de TQueue independientes por par de jugadores.

5.5. Uso de Herramientas de Inteligencia Artificial

En cumplimiento de los requisitos de transparencia establecidos por la cátedra, los autores declaran el uso de herramientas de IA generativa —incluyendo GitHub Copilot, Claude, ChatGPT y Gemini— a lo largo del proyecto, distinguiendo claramente su rol en cada etapa:

Decisiones de arquitectura y diseño. La separación en tres paquetes, el modelo host-árbitro, el mecanismo de `BattlePhase` como máquina de estados, la elección de Q-Learning sobre las alternativas evaluadas y el diseño del vector de características fueron decisiones tomadas íntegramente por los autores mediante análisis y discusión conjunta. Las herramientas de IA fueron utilizadas como interlocutores para sondear alternativas, identificar patrones de diseño comunes en proyectos Haskell de alcance similar y cuestionar puntos de falla de cada propuesta. La decisión final en cada caso fue siempre de los autores.

Implementación. El código fue asistido por herramientas de IA generativa, particularmente para la resolución de problemas de sintaxis de Haskell y la generación de código de estructura repetitiva (*boilerplate*). Todo el código generado fue revisado, cuestionado y comprendido por ambos autores antes de ser incorporado al proyecto. Ambos integrantes pueden explicar y defender cualquier fragmento del código fuente presentado.

Redacción del informe. El contenido del informe fue construido a partir del conocimiento real de los autores, articulado mediante sesiones de preguntas dirigidas en las que las herramientas de IA actuaron como interlocutores estructuradores. La redacción final fue asistida por IA, pero el contenido técnico —decisiones, motivaciones, limitaciones y lecciones aprendidas— es íntegramente propio y verificable contra el código fuente del repositorio.

Referencias

- [1] Haskell.org. *Monad Transformer Library (mtl) Documentation*, 2026. Consultado en línea.
- [2] Graham Hutton. *Programming in Haskell*. Cambridge University Press, 2nd edition, 2016.
- [3] Simon Marlow et al. *The Haskell 2010 Language Report*. Haskell.org, 2010.
- [4] Simon Peyton Jones. Beautiful concurrency. In Andy Oram and Greg Wilson, editors, *Beautiful Code*. O'Reilly Media, 2007.
- [5] Cabal Development Team. *The Cabal User Guide*, 2026. Consultado en línea.